

Local Agent Pipeline Specification

Transitioning from Single-Task Translation to Multi-Phase Orchestration (Phases 1-3)

This document details the architectural modifications required to evolve the existing **LLM-Translator** codebase into a multi-phase, localized agent orchestration pipeline. This restructuring adheres to the blueprints established across Phase 1, Phase 2, and Phase 3 specifications.

1. Data Ingestion: "WordWall" Integration

The previous architecture relied on an isolated, manual file path targeted via user input fields. The updated configuration shifts to a decentralized local directory data layout:

- **Directory Target:** Establish a dedicated folder named **WordWall/** at the root level of your project directory structure.
- **Automated Scan:** Refactor the file verification mechanism to read all prepared files dynamically using `os.listdir()` or similar file-system scanner filters looking specifically for `.xml` formats.
- **Pre-processed Context:** These files serve as a primary reference injection point, supplying predefined baseline parameters, tokens, or local entities into the orchestration cycle prior to running any heavy inference tasks.

2. Intent Categorization & Task Routing

The system must stop assuming all requests are translation tasks. The orchestrator must parse incoming text fields against a strict three-tier classification matrix:

1. **Direct Coding Tasks:** Explicit statements instructing the ecosystem to generate, modify, scaffold, or debug computer programs (e.g., *"Write a Python API routine"*, *"Debug this method loop"*).
2. **Coding-Adjacent Tasks:** Conceptual, structural, or explanatory engineering inquiries (e.g., *"Explain how async threading patterns work"*). These are **handled natively** by the orchestrator and are never delegated downstream.
3. **Non-Coding Tasks:** Standard data organization, document summarization, general logic assistance, or basic translations. These remain entirely within the orchestrator context.

CRITICAL CONTEXT GATHERING PROTOCOL

When a **Direct Coding Task** is successfully identified, the orchestrator must execute a progressive inquiry checklist. It must isolate and query the user for missing variables **one structural question at a time**. It is explicitly forbidden to hypothesize or hallucinate structural parameters like environment language, package targets, scopes, or technical barriers.

3. Phase 3 Integration: The Bridge File Specification

Once a Direct Coding Task has passed validation and its architectural scope is resolved, the orchestrator must not generate the program code itself. Instead, it must cleanly serialize and write out a unified machine-readable JSON transfer token.

Export Metrics & Constraints

- **Strict File Location:** The output token must save precisely to `~/local_agent/prompt_dump.json` (resolving to the absolute user home path). No timestamps, random indexes, or version tags are permitted in the filename. Every new coding event must write-overwrite this clean destination target directly.
- **Downstream Safety Check:** The file is processed completely outside of the web server loop by an independent executor engine (e.g., a decoupled coder script launching a localized Qwen2.5-Coder-7B architecture tier).

Target Serialization Schema

The structural layout of the JSON file must map directly to this precise format:

```
{
  "task_id": "YYYYMMDD_HHMMSS",
  "language": "Python | C# Console | C# WPF | C# MAUI",
  "task_type": "scaffold | boilerplate | debug | assess",
  "output_scope": "method | class | module | fix",
  "existing_code": "none_or_pasted_source_code_as_a_single_string",
  "constraints": [
    "Constraint rule statement 1",
    "Constraint rule statement 2"
  ],
  "user_request": "Structured machine-optimized prompt execution instructions",
  "status": "READY_FOR_CODER"
}
```

MANDATORY KEY CONSTRAINT

The "status" property value must match exactly: **READY_FOR_CODER**. Downstream compilation scripts utilize string-exact comparisons during automated execution passes; arbitrary whitespace modifications or semantic variations will cause pipeline runtime failures.

4. UI Adaptation Matrix (index.html)

To mirror these foundational backend changes, modify the web frontend configuration:

- **Task Inputs:** Repurpose the translation-centric "XML File Path" label to a clean, multi-line "User Workspace / Prompt Input" module.

- **Environment Logs:** Update the status polling routine (`/api/status`) to stream context-appropriate pipeline processing state logs to the frontend display console, substituting older translation metrics with structural states such as: [Phase 1] Analyzing Intent, [Phase 2] Extracting Scope, and [Phase 3] Generating Bridge Payload.